The Google PageRank Score

Megan Millet

December 2024

1 Introduction

Networks are present throughout the real-world; modeling everything from social interactions on social media to the web of interconnected documents and resources that form the internet itself. These are often represented as directed graphs, where nodes in the graph correspond to individual entities and the edges of the graph capture the relationships and connections between nodes. Currently, there are many different ways of studying networks, such as for computing minimum distances, predicting the growth of neurons in brain activity, and determining a *central* or most *important* node of the graph. This notion of *graph centrality* will be the main focus of this paper in constructing the PageRank Algorithm.

Quantifying the *importance* of a node within a directed graph and network is a fundamental problem in graph theory, with applications ranging from search engines to social network analysis and biological systems. There is not one single way of determining the importance of a node; rather, this is a rich topic on its own in the way that the centrality of a graph can have many different interpretations depending on the connectivity of the graph and the system that the graph represents. This paper will begin with an in-depth analysis of various topics of graph centrality, especially in terms of how each centrality approach or algorithm pertains to different graphs with specific underlying assumptions.

An interesting notion of centrality that will be discussed is the *Katz-Bonacich* centrality score. This is crucial to capturing the influence of nodes in the network, while considering a special case of nodes called *dangling nodes*. Moreover, this will be utilized in the construction of the Google PageRank score, which is of primary interest. The PageRank score is itself a centrality quantifier for nodes, but has been continuously modified to accommodate more types of graphs. For example, graphs may have problems of *cycles* or *rank sinks*, which is explored throughout the paper. These challenges have led to some modifications of the PageRank score by using *Markov Chains*.

Markov Chain theory serves as the foundations for the final form of the PageRank score: the Google Matrix. The Google Matrix will resolve all issues of dangling nodes, rank sinks, and even accommodate web surfer *teleportation*. This final formula incorporates stochastic processes to stabilize rankings, thus convergence and existence of the PageRank Algorithm is guaranteed in large-scale networks. In this paper, the mathematical foundation behind the Google PageRank score will be explored, culminating in the construction of the final, and most modern, Google PageRank Algorithm.

2 Background

2.1 Definitions and Notation

We will use the notation $[n] = \{1, 2, \ldots, n\}$ to describe a finite set of $n \in \mathbb{N}$ elements. We will use this notation extensively in the following definitions and throughout the paper.

We will first begin with a fundamental definition of a directed graph, which will describe the networks of interest in our Google PageRank Algorithm.

Definition 2.1. (Directed Graph)
A *directed graph* $G = (V, E)$ is defined by a *vertex set* $V = \{v_i : i \in [n]\}$ where each v_i represents a *vertex*, and an *edge set* $E \subseteq V \times V = \{v_i v_j : v_i, v_j \in V, j \neq i\}$. Edges in this graph are *directed*, where the origin and target of the edge is distinguished. See Figure 1.

Definition 2.2. (Directed Path)
We define a *directed path* in a graph as a finite, ordered sequence of directed edges which join a sequence of vertices, such that all of the edges point in the same direction. A directed graph is called *connected* if there exists a directed path between every pair of vertices v_i and v_j.

Definition 2.3. (Cycle)
A *cycle* in a graph a path that begins and ends at the same vertex, without repeating any edges. See Figure 2.

Definition 2.4. (Degree)
The *degree* of a vertex v_i in an undirected graph is the number of edges involving v_i. By convention, we label the degree of a node v_i by $deg(v_i)$. In a directed graph, we distinguish the *out-degree* of a vertex v_i, defined as the number of edges originating from v_i. We denote this by $deg_{out}(v_i)$, and to avoid a division by zero error in the algorithm, we adopt the convention $deg_{out}(v_i) = 1$ if node v_i has out-degree equal to zero.

Definition 2.5. (Adjacency Matrix)
For a graph $G = (V, E)$, such that $V = \{v_i : i \in [n]\}$, the *adjacency matrix* A is an $n \times n$ square matrix such that:

$$A_{ij} = \begin{cases} 1, & \text{if } v_i v_j \in E \\ 0, & \text{otherwise} \end{cases}$$

That is, it has entry value 1 at coordinate (i, j) if there exists an edge between vertices v_i and v_j in the graph G.

The adjacency matrix is central to representing the graph structure and are important in our formulations of eigenvector centrality scores as we will see in further sections.

Definition 2.6. (Transition Matrix)
Let $G = (V, E)$ be a graph (which can be directed), with an ordered vertex set $V = \{v_i : i \in [n]\}$. Let A be the adjacency matrix of G. We define the transition matrix T such that:

$$T_{ij} = \begin{cases} \frac{A_{ji}}{deg(v_j)} & \text{if } deg(v_j) \neq 0 \\ 1 & \text{if } deg(v_j) = 0 \end{cases} \tag{2.1}$$

For a directed graph, the transition matrix can be modified with out-degrees:

$$H_{ij} = \begin{cases} \frac{A_{ji}}{deg_{out}(v_j)} & \text{if } deg_{out}(v_j) \neq 0 \\ 1 & \text{if } deg_{out}(v_j) = 0 \end{cases} \tag{2.2}$$

Definition 2.7. (Dangling nodes)
A *dangling node* is a vertex that has no out-going edges. In the case of a directed graph modelling a network, a dangling node can be interpreted as a webpage that has no out-going hyperlinks.

Definition 2.8. (Random Walk on a Graph)
A random walk on a graph $G = (V, E)$ is a random process that begins at $v_1 \in V$ which generates an infinite path on G subject to the following rule: the next vertex v_{i+1} is chosen at random from among neighbors of v_i. Each of these neighbors is equally likely to be chosen, and if v_i has no neighbors, then $v_{i+1} = v_i$.

These graph definitions establish a framework for analyzing graph structures and processes of interest to our PageRank Algorithm.

We will now introduce the concept of eigenvalues and eigenvectors, which will be of great use to various eigenvector centrality scores when constructing the PageRank Vector Centrality score and comparing it with other scores.

Definition 2.9. (Eigenvalue and Eigenvector)
We define an *eigenvalue* as $\lambda \in \mathbb{R} \setminus \{0\}$. Given an $n \times n$ matrix A, we can define the *eigenvector* associated to the eigenvalue λ as a nonzero vector $v \in \mathbb{R}^n$ which satisfies the following system:

$$Av = \lambda v$$

Definition 2.10. (Right and Left Eigenvectors)
Eigenvectors can be classified based on their orientation relative to the matrix A. A vector $x \in \mathbb{R}^n$ is a *right eigenvector* if it satisfies:

$$Ax = \lambda x \tag{2.3}$$

where λ is the corresponding *right eigenvalue.*
Alternatively, a vector $y \in \mathbb{R}^n$ is a *left eigenvector* if it satisfies:

$$(y^T A = \lambda y^T) \equiv (A^T y = \lambda y) \tag{2.4}$$

Here λ is the associated *left eigenvalue.*

Definition 2.11. (Positive Vector)
A vector $q = (q_1, q_2, \ldots)^T \in \mathbb{R}^n$ is *positive* if all its components satisfy $q_i > 0$ for all $i \in [n]$. We denote this compactly by $q > 0$.

2.2 Illustrations of Select Definitions

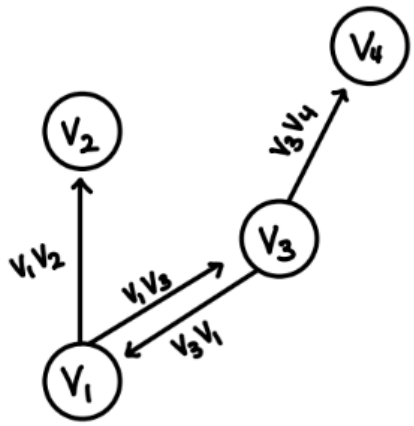

Figure 1: Directed Graph

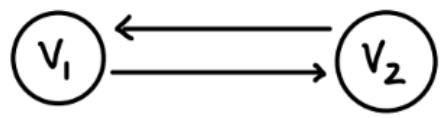

Figure 2: A Cycle Between Two Nodes

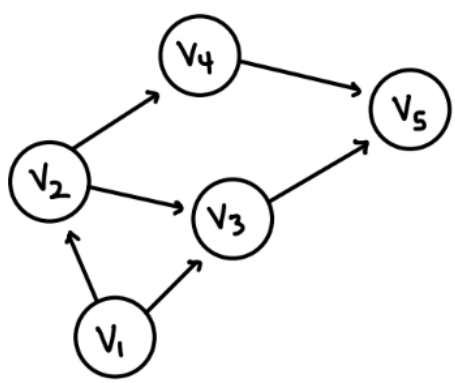

Figure 3: v_5 is a Dangling Node

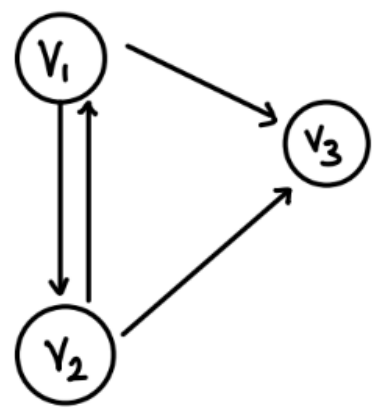

Figure 4: v_3 is a Rank Sink

3 Eigenvector Centrality

3.1 Motivation

In a real-world context, we utilize directed graphs to describe *networks*, which are systems of interconnected objects. In a graph network, some nodes are more *central* or *important* than others. A node which is well-connected, essential, or influential in a network is described as *important*. The aim of Eigenvector Centrality is to quantify the *importance* of a node by not only considering the connections that the node has, but also the relevant importance of the nodes. This recursive idea will be explored and used to define the Google PageRank Algorithm in coming sections.

3.2 Various Eigenvector Centralities

Before discussion various eigenvector centralities, it is important to define the *centrality vector*, which will have different values based on the chosen centrality score.

Definition 3.1. (Centrality Vector)
Let $G = (V, E)$ be a graph. A *centrality vector* q is an $n \times 1$ dimensional vector:

$$q = \begin{pmatrix} q_1 \\ q_2 \\ \vdots \\ q_n \end{pmatrix}$$

such that each entry q_i represents a centrality score of node $v_i \in V$.

Note that each of the centrality scores that we will describe has its own method of calculating the values of q_i, so the exact value of the centrality vector will depend on the chosen score. However, in all centrality scores, higher values of q_i's in q indicates that the nodes are more *important* or *central* when compared with other nodes in the given network.

We will now begin our exploration of different eigenvector centralities by describing the Random Walk Eigenvector Centrality. The Random Walk Eigenvector Centrality quantifies the probability of a random walker being at a node in a strongly connected graph. This probability vector can be found using the largest eigenvalue of the Transition matrix. We will now define it in a mathematical sense:

Definition 3.2. (Random Walk Eigenvector Centrality of a Vertex) Let $G = (V, E)$ be a strongly connected graph and let $\rho = (p_1, \ldots, p_n)$ be a unique probability vector, such that the random walk eigenvector centrality of vertex v_i is p_i. This centrality vector is defined as the unique solution to:

$$T\rho = \rho$$

where T is the transition matrix. Alternatively, if A is the adjacency matrix of the graph G, then ρ satisfies the iterative algorithm for each time-step k:

$$A\rho^{(k+1)} = \lambda\rho^{(k)} \tag{3.1}$$

where λ is the largest eigenvalue of A.

Note that (3.1) satisfies the *right eigenvector* relation (2.3).
We will now investigate a centrality which emphasizes that a node's importance is proportional to the sum of the importance of the nodes pointing to it. This recursive dependency reflects a node's importance ranking in a network through its incoming connections.

Definition 3.3. (Bonacich Centrality of a Vertex) Let $G = (V, E)$ be a directed graph. Let the vector π_i denote the importance of a vertex v_i in terms of its degree in the graph (thus each entry is non-negative), let A be the adjacency matrix of the graph G, and let $\lambda \in (0, \infty)$ be the largest eigenvalue of A. A vertex's importance is proportional to the sum of its neighbors by the following equation:

$$\pi_i = \frac{1}{\lambda} \sum_{j=1}^{n} A_{ji}\pi_j \tag{3.2}$$

The iterative algorithm for the centrality vector π at each time-step k is:

$$\lambda\pi^{(k+1)} = A^T\pi^{(k)} \tag{3.3}$$

COMPARE TO Definition 3.2.

Remark. In some literature, the Bonacich Centrality score is referred to as Eigenvector Centrality.

Note that we use A_{ji} instead of A_{ij}. In directed graphs this subtlety is important as it means that the vertex's centrality is derived from the centrality of the other vertices which point to it. Furthermore, we can see that (3.3) satisfies the *left eigenvector* of (2.4).

3.3 Katz-Bonacich Centrality

We will now define the Katz-Bonacich Centrality score, which extends the Bonacich Centrality score by introducing an external factor to the network. This external factor can help capture indirect influences to the importance of nodes by diminishing the importance of nodes that are further away in the graph. In the PageRank Algorithm, Katz-Bonacich Centrality's principles are adapted to ensure that certain webpage's will not have inflated importance simply because they are linked to important webpages.

Definition 3.4. (Katz-Bonacich Centrality of a Node) Let $x > 0$ be a positive vector (SEE Definition 2.11), β a scalar, and $\lambda \in \mathbb{R} \setminus \{0\}$ be the largest eigenvalue such that

$$x_i = \frac{1}{\lambda} \sum_{j \neq i} A_{ji} x_j + \beta \tag{3.4}$$

Here we call $x = (x_1, \ldots, x_n)^T$ the vector of Katz-Bonacich centralities with parameter α.

The choice of λ being the largest eigenvalue of the system is important as λ corresponds to the most dominant importance or influence within the graph. By dividing each node by λ, we are able to control how much influence a node receives from its neighbors or in-coming nodes. This allows us to consider indirect connections (such as long paths) in the graph when we calculate the centrality.

The factor β allows the Katz-Bonacich centrality score to capture both direct and indirect influence by accounting for *paths* (SEE Definition 2.2) of arbitrary length in the directed network. Bonacich Centrality instead focuses on the direct influence of immediate neighbors of a node, meaning it disregards longer paths.

4 Google's PageRank Algorithm

4.1 Motivation

The Google PageRank algorithm seeks to rank pages based on their importance in the context of the entire web. Here, we represent the web as our graph of interest, vertices as each webpage in the web, and edges are represented by hyperlinks between webpages. In this setting, having a high PageRank score indicates that a webpage has many visitors (high-traffic), is influential to the web, and will appear at the top of search results.

The PageRank algorithm is built off of the Katz-Bonacich Centrality score, but modifies it to overcome the limitation that the Katz-Bonacich centrality assigns high-importance to any webpage which is linked by an important or high-traffic webpage [Mey23]. This could lead to misleading results. For example, the Marty the Martlet McGill mascot information page would have a high-importance ranking simply because it links to the highly important McGill University Homepage

website. To avoid inflating the importance of niche subjects, the PageRank algorithm has created additional adjustments along the way which account for these cases.

4.2 The Original Construction

Initially, the PageRank algorithm was conceptualized as a centrality score which was solely based on the idea that a page's importance could be determined by importance of pages linking to it [BP98]. We have the following construction:
Let $G = (V, E)$ be our Google web-graph. Let $r = (r_1, \ldots, r_n)^T$ be the centrality vector where r_i is the pagerank score (importance) of vector v_i. Then we define the original PageRank algorithm as:

$$r_i = \sum_{j=1}^{n} \frac{A_{ji}}{deg_{out}(v_j)} r_j \tag{4.1}$$

However, this model soon encountered many challenges. Pages that had no outgoing hyperlinks (think of a dead-end or *dangling node*) would trap ranking scores, which skews the centrality scores and disrupts the overall ranking structure [LM12]. Thus, more modifications were needed to improve upon this algorithm.

4.3 Construction Using Katz-Bonacich Centrality

The failures of the original model motivated the new construction which is based off an adjustment to the Katz-Bonacich Centrality score. The main adjustment that is introduced by the PageRank Algorithm is to normalizing each page's influence by its out-degree, thereby ensuring that a node's influence score is proportional to the number of links that it offers. Thus, it limits the effect of nodes with many outbound links.

Remark. Note that the following algorithm is in the case where the directed graph has no *dangling nodes* (SEE Definition 2.7).

Definition 4.1. (PageRank Score of a Node) Let $G = (V, E)$ be a directed graph. Let A be an $n \times n$ adjacency matrix associated to G, and $\alpha \in [0, 1)$ denote the damping factor. We denote the *PageRank score* r_i of a node v_i as:

$$r_i = \alpha \sum_{j=1}^{n} \frac{A_{ji}}{deg_{out}(v_j)} r_j + 1 \quad \forall i \in \mathbb{N} \tag{4.2}$$

We can write this in as an interative process with respect to each time-step k as:

$$r^{(k+1)} = \alpha H r^{(k)} + \mathbf{1} \tag{4.3}$$

where H is the *modified transition matrix* which satisfies (2.2). Here we have that $r = (r_1, r_2, \ldots, r_n)^T$ is a centrality vector. We also call H the *hyperlink matrix* of the equation.

More concretely, the iterative process for computing the centrality vector r of PageRank scores is:

- Start with $r^{(0)} = \mathbf{1}$ (equal importance for all nodes) for time-step k=0.
- Update at each step: $r^{(k+1)} = \alpha H r^{(k)} + \mathbf{1}$

- Repeat until $||r^{(k+1)} - r^{(k)}|| < \epsilon$.

Remark. The key difference here between the PageRank score and the Katz-Bonacich centrality score is by the normalization using out-degrees. The PageRank score normalizes the adjacency matrix by the out-degree of each node, which prevents pages with many out-links from disproportionately affecting the importance of other pages.

4.3.1 The α Parameter

What exactly does the α parameter represent? We define α as the probability that a *random web surfer* follows the hyperlinks within a given webpage instead of closing the current page or opening a new page that is not linked to the current [LM12]. That is, if $\alpha = 0.3$ then the user has a 30% chance of clicking a hyperlink on the current webpage, and a 70% chance of *teleporting* or leaving the page entirely.

In the PageRank Algorithm, $\alpha \in [0, 1)$ is a free parameter that can be chosen to "tune" the algorithm. The higher the value of α, the more importance the algorithm will place on indirect links rather than direct links. By convention, Google chooses $\alpha = 0.85$ [BP98].

We would now like to explore how α is chosen in practice. Choices of α are empirical in nature, because the parameter represents human behavior while browsing the web [BSV07]. For example, if $\alpha = 0$, this would annihilate the webpage process from Definition 4.1 resulting in a trivial equation. Also, as α gets closer to 1, the influence that the web part contributes to (4.3) becomes more and more important [CG09]. Some studies have been conducted to choose α based on human behavior, the location of hyperlinks on the webpage, the type of webpage, and many other ways. For simplicity, $\alpha = 0.85$ is assumed throughout this paper.

4.3.2 Choice of $\beta = 1$

As mentioned earlier in the Katz-Bonacich centrality score, the β term weighs the contribution of paths based on their lengths [Mun17]. In the PageRank Algorithm, we have that $\beta = 1$. This assumes that all paths in the network, regardless of their length, contribute equally to the importance of the node. Instead of continuously modifying the β term, the PageRank Score uses α and the out-degree normalization to control the importance of nodes and add a base-line importance to dangling-nodes.

This, however, comes at a cost to the precision of the centrality computation: By setting $\beta = 1$, longer paths in the network have equal importance to shorter paths. If we consider a path with 100 nodes, say $\{v_1, v_2, \ldots, v_{100}\}$ then the path from v_1 to v_2 is of equal importance to the ranking of v_1 and the path from v_1 to v_{100}. In the context of the PageRank score, this means that a webpage that is 100 hyperlinks away from the original page will contribute equally to the importance of the original webpage as a webpage that is one hyperlink away. This creates inconsistent results with reality. Thus, we seek to create an algorithm that will still consider dangling nodes, rank sinks, and cycles, but without applying arbitrary importance to long paths; we want there to be some *decay* in the influence that far away nodes provide toward nodes at the beginning of any given path.

5 Markov Chain Theory and The Google Matrix

5.1 Motivation

While (4.3) solved the problem of skewed ranking scores being applies to niche webpages, there are still some problems left in the Katz-Bonacich score centrality formulation. For example, *cycles* between webpages can cause the ranking process to become trapped in an infinite loop, causing the rankings to inflate infinitely (See Figure 2). Since the Katz-Bonacich centrality score sums the contributions from all paths and then normalizes these contributions, an infinite cycle was not accounted for in this formulation. Furthermore, another common issue to arise in a network are *rank sinks*. A rank sink is a node that accumulates more and more PageRank at each iteration of the algorithm, most commonly if there is a cycle pointing to this dead-end node (See Figure 4). Both of these situations have nodes with infinitely growing PageRank score, and thus will be addressed in the next model: The Google Matrix.

Before we are able to discuss the Google Matrix, we need to build the necessary theory behind the algorithm to better explain why this solves the previously discussed problems. To do so, we require some Markov Chain theory. We will then discuss the new parameters involved in this new algorithm, and conclude the section with an overview of the Google Matrix and some interesting properties associated with it.

5.2 Intro to Markov Chains

We will first begin by giving a brief overview of Markov Chains and why this topic motivates our construction of the Google Matrix.

In the context of the PageRank Algorithm, we will think of the process of someone surfing the web and clicking links from page to page. Here, the *state space* would be the set of all web pages, and the *random variable* X_t is the current webpage that is being viewed at time t. We will now define this formally as a *stochastic process*:

Definition 5.1. (Stochastic Process)
A *stochastic process* is a set of random variables $\{X_i\}_{t=0}^{\infty}$ which have a common range $\{s_1, s_2, \ldots, s_n\}$, which is called the *state space* for the process. We think of X_t as the state of the process at time t.

Stochastic processes provide a flexible framework for modeling systems that evolve randomly over time. However, the random movement of a web surfer can be more structured as we will see further in this section. This brings us to the concept of a *Markov Chain*, which imposes an important property on stochastic processes: the future state depends only on the current state, not the full history of states. We now define this formally:

Definition 5.2. (Markov Chain)
A *Markov Chain* is a stochastic process (SEE Definition 5.1) that satisfies the Markov Property:

$$\mathbb{P}(X_{t+1} = s_j | X_t = s_{i_t}, X_{t-1} = s_{i_{t-1}}, \ldots, X_0 = s_{i_0}) = \mathbb{P}(X_{t+1} = s_j | X_t = s_{i_t}) \tag{5.1}$$

for each $t \in [n]$. We call a Markov chain *aperiodic* if for every state s_i, the greatest common divisor of all possible return times to state s_i is 1. That is, aperiodicity means that the chain does not exhibit strictly cyclic behavior.

The Markov Property (5.1) implies that the process is *memoryless* in the sense that the future state of the chain at the next time period depends only on the current state, not on the past history [PB02].

We will also quickly define an *irreducible matrix* and some vectors which will be important in our proofs later:

Definition 5.3. (Probability and Stationary Vectors)
A *probability vector* v is satisfies the following two conditions:

(i) $v_i \geq 0 \quad \forall i \in \mathbb{N}$

(ii) $\sum_{i=1}^{n} v_i = 1$

A *stochastic matrix* is an $n \times n$ matrix which has probability vectors as columns.
Let A be an $n \times n$ stochastic matrix. A *stationary vector* $v \in \mathbb{R}^n$ is a probability vector which satisfies:

$$Av = v$$

Definition 5.4. (Irreducible Matrix)
A stochastic matrix is *irreducible* if it represents a Markov chain in which every state is reachable from every other state.

We can consider the process of surfing the web as a Markov Chain, provided that the next page that the web surfer visits does not depend on the pages that were visited in the past; it only depends on the current page. That is, if a web surfer randomly clicks a link on the current page to go to the next page, it is a Markov chain. We can call this a *random walk* on the hyperlink structure of a network.

To describe this process mathematically and compute the likelihood of a surfer moving between pages, we need a way to quantify the probabilities associated with moving from one state (webpage) to another. This leads us to the concept of the *transition probability matrix*:

Definition 5.5. (Transition Probability Matrix)
The *transition probability* $p_{ij}(t) = \mathbb{P}(X_t = S_j | X_{t-1} = S_i)$ is the probability of being in a state S_j at time t given that the Markov chain is in state S_i at time $t-1$. This can be interpreted as the probability of moving from S_i to S_j at time t. We can then construct the *transition probability matrix* $P(t) = [p_{ij}(t)]$. The sum of each row in the matrix is equal to 1, so we also describe it as a *stochastic matrix*.

Example. In the PageRank Algorithm suppose we have a network consisting of three webpages that are all connected: $\{v_1, v_2, v_3\}$. Let us suppose that the web surfer begins on webpage v_1. The probability that this web surfer remains on webpage v_1 or moves to webpages v_2 or v_3 can be represented as entries in a transition probability matrix. For illustrative purposes, the transition probability matrix for this network could be:

$$P = \begin{bmatrix} 0.1 & 0.3 & 0.2 \\ 0.5 & 0.2 & 0.3 \\ 0.4 & 0.5 & 0.5 \end{bmatrix}$$

Here, each row represents the current page (v_1, v_2, v_3) that the surfer is on, and each column represents the destination page that the web surfer may travel to. So, $p_{12} = 0.3$ represents that the web surfer has a 30% chance of transitioning from v_1 to v_2 in one step.

Furthermore, we have already seen a transition probability matrix in Section 4.3:

Proposition 5.6. In (4.3), our *hyperlink matrix* H is a transition probability matrix.

Proof. To prove that the hyperlink matrix H is a transition probability matrix, we need to prove the following:

(1) All entries are non-negative. ($H_{ij} \geq 0$)

(2) Each column of H sums to 1. ($\sum_{i=1}^{n} H_{ij} = 1$)

By definition of the adjacency matrix A, $A_{ji} \geq 0$ (SEE Definition 2.5). From the definition of out-degrees, $deg_{out}(v_j) > 0$ implies that $\frac{1}{deg_out(v_j)} > 0$. Hence,

$$H_{ij} = \frac{A_{ji}}{deg_{out}(V_j} \geq 0$$

Thus we have that all entries of H are non-negative. Now we will prove that each column of H sums to 1. We have that:

$$\sum_{i=1}^{n} H_{ij} = \sum_{i=1}^{n} \frac{A_{ji}}{deg_{out}(v_j)} = \frac{\sum_{i=1}^{n} A_{ji}}{deg_{out}(v_j)}$$

Recall that $A_{ji} = 1$ if there exists a directed edge from vertex v_j to vertex v_i, and 0 otherwise. This implies that $\sum_{i=1}^{n} A_{ji}$ counts the total number of edges leaving vertex v_j, which is equivalent to $deg_{out}(v_j)$ (SEE Definition 2.4). Thus, we have:

$$\sum_{i=1}^{n} H_{ij} = \sum_{i=1}^{n} \frac{A_{ji}}{deg_{out}(v_j)} = \frac{\sum_{i=1}^{n} A_{ji}}{deg_{out}(v_j)} = \frac{deg_{out}(v_j)}{deg_{out}(v_j)} = 1$$

Hence we have that each column of H sums to 1, thus we have that H is a transition probability matrix. □

Remark. We assumed that our directed graph has no *dangling nodes* (SEE Definition 2.7) for our out-degree normalized PageRank Algorithm.

In the next section, we will be able to handle the case where the directed graph does contain dangling nodes, by a modification to this hyperlink matrix H. Then, we will use this to motivate an important theorem:

Theorem. A unique positive PageRank vector exists when the Google matrix is stochastic and irreducible. Furthermore, with aperiodicity the iterative algorithm will converge to this PageRank vector regardless of the starting vector.

We will prove this theorem after introducing the Google Matrix in its entirety.

5.3 Adjustments to the PageRank Algorithm

With this hyperlink matrix H, we can see how the PageRank Algorithm naturally incorporates Markov Chain theory. In this section, we will account for cycles and rank sinks in our further adjustments to the PageRank Algorithm by first modifying the hyperlink matrix with a *stochasticity adjustment*. Furthermore, we will define the Teleportation Matrix and analyze its improvements to the previous model and conclude by using both these matrices to define our final model: The Google Matrix.

5.3.1 The Stochasticity Adjustment to the Hyperlink Matrix

Our modification to the hyperlink matrix will solve the case when a web surfer reaches a *dangling node* (which is a webpage with a dead-end), and allow the web surfer to *teleport* to a different web page. By teleporting, we refer to a web surfer exiting the page and entering a new webpage which had no connection or hyperlink to the previous webpage.

In our network, there are unavoidable dangling nodes which cause rows of zeros to appear in the hyperlink matrix (as these nodes will have no out-going hyperlinks). Our modification to H will replace the $\mathbf{0}^T$ rows of zeros with $\frac{1}{n}\mathbf{e}^T$, where $\mathbf{e}$ is the standard basis vector and n represents the total number of nodes in the network. As a result of this modification, the random surfer will now be able to randomly teleport to any other webpage in the network with equal probability dependent on the number of nodes in the network. This modification makes H *stochastic* in the general directed graph setting [LM12].

We will write this stochasticity adjustment mathematically as a new matrix S:

$$S = H + \delta_d(\frac{1}{n}\mathbf{e}^T)$$

where δ_d is a vector such that

$$\delta_i = \begin{cases} 1, & \text{if } v_i \text{ is a dangling node} \\ 0, & \text{otherwise} \end{cases}$$

Proposition 5.7. S is a transition probability matrix for a Markov chain.

Proof. We will prove that S is a transition probability matrix by verifying the following properties:

(1) All entries of S are non-negative.

(2) Each row of S sums to 1.

We will begin by verifying all entries of S are non-negative. By definition, H has all non-negative entries. We also have that $\delta_d \in \{0, 1\}$ and $\frac{1}{n}\mathbf{e}^T$ is a row vector with all entries equal to $\frac{1}{n}$. Thus $S = H + \delta_d(\frac{1}{n}\mathbf{e}^T)$ has non-negative entries ($S_{ij} \geq 0 \quad \forall i, j \in \mathbb{N}$).

Now we will verify that each row of S sums to 1. We will consider two cases of vertices:

Case 1: The vertices v_i are dangling nodes:

$$\sum_{i=1}^{n} S_{ij} = \sum_{i=1}^{n} H_{ij} + 0 = 1$$

Where $\sum_{i=1}^{n} H_{ij} = 1$ follows from Proposition 5.6.

Case 2: The vertices v_i are dangling nodes:

$$\sum_{i=1}^{n} S_{ij} = \sum_{i=1}^{n} H_{ij} + 1(\frac{1}{n}) = \sum_{i=1}^{n} 0 + \frac{1}{n} = n(\frac{1}{n}) = 1$$

All of the entries of $H_{ij} = 0$ for dangling nodes v_i, and $\delta_d = 1$ for dangling nodes.
Thus we have that in both cases, each row of S sums to 1, so S is a transition probability matrix. □

We now have that S is a transition probability matrix in our network that includes dangling nodes, but this is still not enough to prove our desired Theorem.

5.3.2 The Primitivity Adjustment of The Teleportation Matrix

The concept of primitivity is essential to prove that the Google Matrix algorithm will indeed converge uniquely to the PageRank vector. In the context of Markov chains, primitivity ensures that the chain is *irreducibe* (every state can be reached from any other state) and *aperiodic* (the chain does not cycle between disjoint subsets of states) [PB02]. We will begin by defining what a *primitive matrix* is, and then go over how its properties affect our overall algorithm.

Definition 5.8. (Primitive Matrix)
An $n \times n$ matrix M is called *primitive* if it is non-negative ($M \geq 0$) and $M^k > 0$ for some $k \in \mathbb{N}$. That is, it is aperiodic and stochastic.

We will now motivate a new primitive matrix called the Teleportation Matrix, which will be incorporated in our new Google Matrix.

While web surfing, there are times when someone closes their current page and opens a new page by typing the new URL directly or opening a new tab. We call this teleporting, in the sense that the web surfer travels to a new webpage in the network which has no connections to the previous webpage.
We define the Teleportation Matrix E as:

$$E = \frac{1}{n}\mathbf{e}\mathbf{e}^T$$

Here we have that teleporting within a network is random, as each node in the network has an equal probability of being picked when a websurfer moves to a different page without following a hyperlink. Furthermore, we will show that E is stochastic:

Proposition 5.9. The teleportation matrix E is *stochastic*.

Proof. To prove that E is stochastic, we will show that each row sums to 1 and it has entries that are between 0 and 1. First, it is clear that the entries of E are between 0 and 1 as $E_{ij} = \frac{1}{n}$. Now, we show that the rows sum to 1:

$$\sum_{j=1}^{n} E_{ij} = \sum_{i=1}^{n} \frac{1}{n} = n(\frac{1}{n}) = 1$$

Thus we have that E is *stochastic*. □

We now have the tools needed to define the Google Matrix.

5.4 The Google Matrix

We denote the Google Matrix by G such that:

$$G = \alpha S + (1 - \alpha)E$$

As before, $\alpha \in [0, 1)$ is a parameter which controls the proportion of time that a random surfer follows the hyperlinks of a webpage rather than teleporting to a new page.

We will now go over some immediate properties of the Google Matrix G that follow from our construction using the stochastic hyperlink matrix S and the teleportation matrix E.

Proposition 5.10. We now present key properties of the Google Matrix G:

(i) G is stochastic

(ii) G is irreducible

(iii) G is aperiodic

(iv) G is primitive

Proof. (i) To prove that G is stochastic (SEE Definition 5.1), we will verify that all entries are non-negative and each row of G sums to 1. We have that:

$$G_{ij} = \alpha S_{ij} + (1 - \alpha)E_{ij}$$

Recall that both S_{ij} and E_{ij} are non-negative matrices. We also have that $\alpha \in [0, 1)$ and $1 - \alpha \in (0, 1]$, thus $\alpha S_{ij} \geq 0$ and $(1 - \alpha)E_{ij} \geq 0$. So it follows that $G_{ij} \geq 0$. Now, we will show that the each row sums to 1:

$$\begin{aligned}
\sum_{j=1}^{n} G_{ij} &= \sum_{j=1}^{n} \alpha S_{ij} + (1 - \alpha)E_{ij} \\
&= \alpha \sum_{j=1}^{n} S_{ij} + (1 - \alpha) \sum_{j=1}^{n} E_{ij} \\
&= \alpha(1) + (1 - \alpha)(1) \\
&= \alpha + 1 - \alpha \\
&= 1
\end{aligned}$$

Thus we have that G is a stochastic matrix.

(ii) We will now prove that G is irreducible (SEE Definition 5.4). We need to ensure that $(G_{ij})^k > 0$ for all $i, j \in \mathbb{N}$ and some $k \in \mathbb{N}$. We have that:

$$G_{ij} = \alpha S_{ij} + (1 - \alpha)E_{ij} \geq (1 - \alpha)E_{ij} > 0$$

Since $(1 - \alpha) > 0$ and $E_{ij} > 0$. Thus, we have shown that for $k = 1$ $(G_{ij})^k > 0 \quad \forall i, j \in \mathbb{N}$ as desired.

(iii) We will now prove that G is aperiodic (SEE Definition 5.2). Note that each vertex v_i has a non-zero probability of remaining in its current state due to the structure of G. That is:

$$G_{ii} = \alpha S_{ii} + (1 - \alpha)E_{ii}$$

Since we have that $E_{ii} > 0$ for all v_i and $\alpha < 1$, we have that $G_{ii} > 0$. This creates a self-loop at each vertex v_i, meaning that the Markov chain can return to the state v_i in one time step. This eliminates any strict cyclic behavior in G, ensuring that it is aperiodic.

(iv) Since G is aperiodic and stochastic, it is primitive by our definition (SEE Definition 5.8).

□

Now that we have proved some of the properties of the Google Matrix, we can describe the Adjusted Pagerank of a Vector utilizing the Google Matrix:

Definition 5.11. (The Google Matrix Adjusted Pagerank of a Vector)
We define the *Google's Adjusted PageRank Algorithm* as:

$$r^{(k+1)} = r^{(k)}G \tag{5.2}$$

where $r^{(k)}$ denotes the k-th iteration of the PageRank Vector.

Remark. The iterative process used to compute the k-th iteration of the PageRank Vector is the *power method*, which is a computer-based algorithm. Due to the mathematical nature of this research paper, we will not go over the specifics of this algorithm and will assume it is being used when discussing any convergence over time of the algorithm.

Now that we have established some properties of the Google Matrix G and have restated the PageRank Algorithm in terms of G, one important question remains: Can we guarantee that the iterative application of G defined by the Adjusted PageRank Algorithm leads to a meaningful and stable outcome?

To answer this question, we must first prove the famous Perron-Frobenius Theorem, which will give us the necessary tools needed to answer this question.

Theorem 5.12. (Perron-Frobenius Theorem) Let A be an $n \times n$ stochastic, irreducible matrix. Then we have the following properties:

(i) A has a unique largest eigenvalue $\lambda = 1$, which is called the Perron-Frobenius eigenvalue

(ii) The eigenvector corresponding to $\lambda = 1$ is strictly positive

(iii) For any initial *probability vector* $x^{(0)}$, the iterative process:

$$x^{(k)} = Ax^{(0)}$$

converges to the unique *stationary vector* of A (SEE Definition 5.3)

Proof. We now prove each property of the theorem:

(i) We will show that the eigenvalue $\lambda = 1$ exists and is unique. Since A is a stochastic matrix, the sum of the entries in each row is equal to 1. Therefore,

$$A\mathbf{1} = \mathbf{1}$$

for eigenvector $\mathbf{1} = (1, \ldots, 1)^T$ an $n \times 1$ vector of 1's. This eigenvector has associated eigenvalue $\lambda = 1$.

Lemma 5.13. Let A be a stochastic matrix. Then all of its eigenvalues satisfy $|\lambda| \leq 1$.

Proof. Let A be a stochastic matrix with λ an eigenvalue, and $v \neq 0$ be the associated eigenvector of λ. That is, it satisfies:

$$Av = \lambda v$$

We will now consider:

$$\begin{aligned} ||Av|| = \sum_{i=1}^{n} \left| \sum_{j=1}^{n} A_{ij} v_j \right| &\leq \sum_{j=1}^{n} \left(\sum_{i=1}^{n} A_{ij} \right) |v_j| \\ &= \sum_{j=1}^{n} |v_j| \\ &= ||v|| \end{aligned}$$

By the triangle inequality and the fact that $\sum_{i \geq 1} A_{ij} = 1$. Since $||Av|| \leq ||v|| = 1||v||$, the absolute value of each eigenvalue of the matrix must be bounded above by 1. Thus, $|\lambda| \leq 1$ □

Since A is a stochastic matrix, by Lemma 5.13 $\lambda = 1$ is the largest possible eigenvalue of A. We will now show that λ is unique. Since A is an irreducible matrix, $\lambda = 1$ is *simple*, meaning it has geometric multiplicity equal to 1. This implies that there is no other eigenvalue of the same value in A.

(ii) We will now prove that the eigenvector associated to $\lambda = 1$ is strictly positive. Since A is irreducible, $\forall i, j \in \mathbb{N}$ indices and some $k \in \mathbb{N}$, we have that $(A_{ij})^k > 0$. Since $A^k > 0$ and $\lambda = 1$ is simple, any eigenvector associated to it must be strictly positive, as the vector associated to $\lambda = 1$ is the limit of the positive iterates of $A^k x^{(0)}$ starting from any $x^{(0)}$ initial positive probability vector.

(iii) We now prove convergence. Let $x^{(0)}$ be a probability vector. The stochasticity of A ensures that $x^{(k)} = A^k x^{(0)}$ remains a probability vector for all $k \in \mathbb{N}$. By decomposing A into its eigenvectors and eigenvalues, the iterative process $x^{(k)} = A^k x^{(0)}$ converges onto the eigenspace corresponding to $\lambda = 1$. Since $\lambda = 1$ is simple and unique, so is its eigenvector up to scaling. This is exactly the stationary vector v satisfying $Av = v$, such that $x^{(k)} \to v$ as $k \to \infty$.

□

G's properties ensures that it is not only a computational tool but also a well-defined theoretical object. By this we mean that the stochasticity of G guarantees that the total rank is conserved at each time step, and the irreducibility and aperiodic properties guarantee that every page contributes to the final ranking. These properties will enable us to prove our fundamental theorem, which will confirm the existence and uniqueness of the PageRank vector and prove the convergence of the iterative algorithm used to compute the PageRank vector. We now present the theorem:

Theorem 5.14. A unique positive PageRank vector exists when the Google matrix is stochastic and irreducible. Furthermore, with aperiodicity the iterative algorithm will converge to this PageRank vector regardless of the starting vector.

Proof. By the Perron-Frobenius Theorem 5.12, a stochastic and irreducible matrix has a unique stationary vector x^T such that:

$$x^T = x^T G$$

Here we have that x^T is the PageRank vector. This ensures the existence and uniqueness of the PageRank vector. We also have that by aperiodicity, Property (iii) of the Perron-Frobenius Theorem 5.12 guarantees that this iterative process will indeed converge to this PageRank vector. $\square$

6 Conclusion

This research paper demonstrates the critical role that eigenvector centrality and Markov chain theory contributes toward the construction of the PageRank Algorithm. Real-world networks present many challenges including dangling-nodes, rank sinks, and cycles. These challenges motivated the development of the Katz-Bonacich modification to the PageRank Score as well as the construction of the Google Matrix, which stabilizes rankings and guarantees convergence through the Perron-Frobenius Theorem. From this, we were able to construct our final PageRank Algorithm (5.2) by combining all of the tools that were established throughout the paper. A final question arises: How can these concepts and algorithms extend to more than just Google's network?

PageRank inspired algorithms have been used in a wide variety of fields, such as ranking genes or proteins in biology, it can be used in risk-portfolio modeling, and in spectral embedding theories [SHG24]. From a theoretical perspective, there is on-going research to extend the PageRank algorithm to dynamic and temporal networks, where connections between edges also evolve over time. There has been recent research into multi-layered networks that represent systems with different types of connections (e.g. social, familial, and professional ties in human networks). This type of research utilizes a combination of eigenvector centrality with Markov chain theory in a similar construction as the Google Matrix and PageRank Algorithm.

One promising area of research lies in adapting the Google Matrix to heterogeneous systems, where nodes need not represent a single concept. For example, in citation networks the algorithm can be modified to account for fields of study, publication quality, or recency, which would produce rankings that may be more nuanced than traditional centrality methods. Similarily, PageRank-inspired algorithms can be used in e-commerce to customize user preferences, product categories, targeted advertisements, and trends. These applications highlight the flexibility of the algorithm and its potential to address many modern challenges.

Another important extension of the PageRank Algorithm is in data systems. Throughout the past decade, almost all sources of information has been transformed into a digital medium, hence the need for scalable and computationally efficient algorithms to process this amount of data remains a significant area of research. Parallelized and other implementations of PageRank has driven advancements in high-performance computing and machine learning networks. The mathematical principles discussed in this research paper will continue to inspire new solutions across diverse fields and developments.

References

[BP98] Sergey Brin and Lawrence Page. The anatomy of a large-scale hypertextual web search engine. *Computer Networks and ISDN Systems 30(1-7):107-11*, 1998.

[BSV07] Paolo Boldi, Massimo Santini, and Sebastiano Vigna. A Deeper Investigation of PageRank as a Function of the Damping Factor. In Andreas Frommer, Michael W. Mahoney, and Daniel B. Szyld, editors, *Web Information Retrieval and Linear Algebra Algorithms*, volume 7071 of *Dagstuhl Seminar Proceedings (DagSemProc)*, pages 1–19, Dagstuhl, Germany, 2007. Schloss Dagstuhl – Leibniz-Zentrum für Informatik.

[CG09] Paul G. Constantine and David F. Gleich. Random alpha PageRank. *Internet Math.*, 6(2):189–236, 2009.

[LM12] Amy N. Langville and Carl D. Meyer. *Google's PageRank and beyond: the science of search engine rankings*. Princeton University Press, Princeton, NJ, paperback edition, 2012.

[Mey23] Carl D. Meyer. *Matrix analysis and applied linear algebra*. Society for Industrial and Applied Mathematics (SIAM), Philadelphia, PA, second edition, [2023] ©2023.

[Mun17] Kamesh Munagala. Lecture 12: Random walks and graph centrality, 2017.

[PB02] Pierre Priouret Paolo Baldi, Laurent Mazliak. *Martingales and Markov Chains Solved Excercises and Elements of Theory*. Chapman and Hall/CRC, 2002.

[SHG24] Disha Shur, Yufan Huang, and David F. Gleich. A flexible PageRank-based graph embedding framework closely related to spectral eigenvector embeddings. *J. Appl. Comput. Topol.*, 8(5):1417–1444, 2024.